DB Monitoring: обнаружение аномалий в метриках качества данных

Итоговый отчёт по дисциплине «Интеграция искусственного интеллекта в разработку» (Нетология). Репозиторий: aleksandr-novikov/ai-integration-development. Редакция 12.11.2026, сверена с `master` `13b31f3`. Все числа взяты из сохранённых артефактов, ссылки на них приведены в тексте; новые эксперименты для отчёта не проводились.

Участник	GitHub	Зона ответственности
Александр	aleksandr-novikov	Архитектура, MLflow, версии моделей и интеграция
Раиль	rail-ss	Постановка задачи, roadmap, отчёт и презентация
Виталий	vtm9	Тесты, CI, Docker, выпуск и эксплуатация
Оксана	ksdergach	Данные, валидация, API и коллекторы

Роли согласованы командой (CASE.md, раздел 10). Это зоны ответственности, а не оценка личного вклада; исполнители задач указаны в Issues и PR.

1. Бизнес-задача и назначение

Проблема. Нарушения качества данных часто не вызывают технического отказа БД: резко растёт доля NULL, нетипично меняется объём таблицы, ломается привычная динамика. Такие отклонения могут оставаться незамеченными, пока ошибочные данные не попадут в отчётность. Задача проекта — сократить время обнаружения таких инцидентов автоматическим анализом истории метрик (CASE.md).

Пользователь — дата-инженер, аналитик данных или другой специалист, отвечающий за качество данных. Система указывает на подозрительное состояние, но не исправляет данные и не принимает решение вместо человека.

Вход. DB Monitoring периодически (по умолчанию раз в 15 минут) собирает по каждой таблице метрики: число строк, NULL-статистику, распределения, схему. Модель использует четыре признака: `row_count`, среднюю по колонкам долю NULL `null_rate` и их изменения между соседними наблюдениями `d_row_count`, `d_null_rate`.

Выход. Для наблюдения: объект мониторинга, время, значения признаков, признак «норма/аномалия», числовая оценка аномальности, версия модели. Результат показывается на дашборде, доступен через API и используется в уведомлениях.

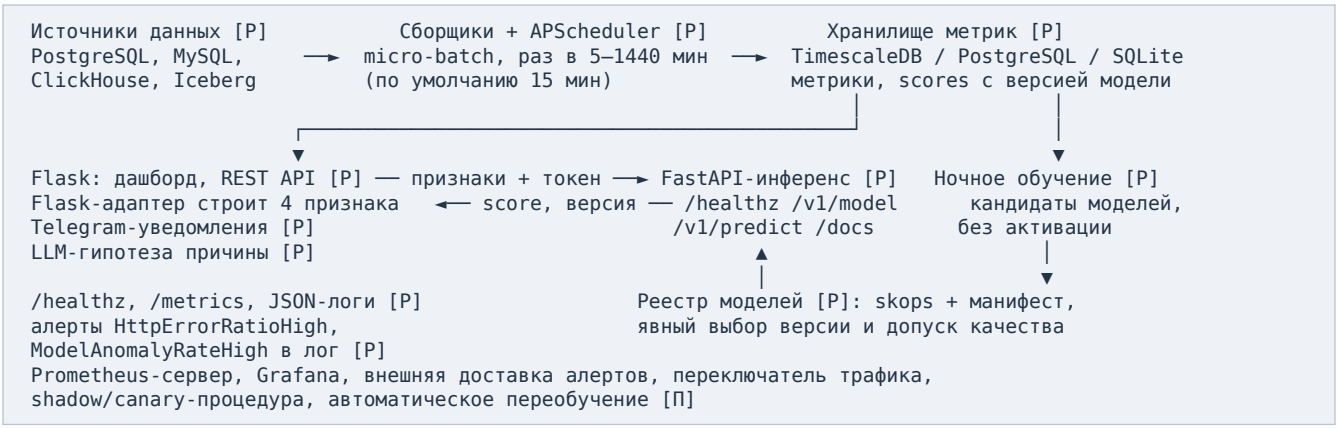
Роль ML. Isolation Forest — метод без учителя: находит нетипичные наблюдения без ручного правила для каждого вида инцидента. Сигнал модели — повод для проверки человеком, а не доказательство ошибки. Детектор унаследован от исходного DB Monitoring (коммит `1a869e4`); в курсе он превращён в воспроизводимый и проверяемый компонент: датасет, валидация, эксперименты, тесты, сервис, контейнеры, мониторинг и стратегия дрейфа.

Критерии успеха зафиксированы до экспериментов (CASE.md, раздел 7): обнаружено ≥ 80 % размеченных инцидентов; ложные сигналы — ≤ 10 % наблюдений вне инцидентов (FPR); медианная задержка обнаружения резких инцидентов ≤ 1 цикла сбора (15 минут).

2. Архитектура и технологический стек

Система — модульный монолит на Flask со встроенным планировщиком и отдельным сервисом инференса на FastAPI (Architecture.md, ADR-001).

FastAPI не читает историю и не обучает модель: он проверяет токен, полный источник `(project, connection, schema, table)` и выбранную неизменяемую версию из реестра (inference/README.md, models/README.md). Три маршрута и Swagger UI проверены настоящим Uvicorn (final-verification.md). Учебные MLflow-эксперименты не публикуют модель автоматически. Схема ниже отмечает реализованные компоненты [Р] и проектируемые [П].



Технология	Почему выбрана
Python, Flask, SQLAlchemy, APScheduler	Переиспользуют рабочие UI, доступ к нескольким СУБД и фоновые задания исходного проекта
TimescaleDB / PostgreSQL, SQLite	Hypertable для временных рядов метрик; SQLite — локальный режим и тесты
scikit-learn: StandardScaler + IsolationForest	Числовые признаки без готовых меток инцидентов; быстрое обучение на одной таблице
MLflow, skops	Протокол экспериментов, параметры, метрики и артефакты; безопасная сериализация моделей реестра
FastAPI, OpenAPI 3.1	Отдельная проверяемая граница инференса со Swagger UI; контракт общий для сервиса и адаптера
pandas	Явные проверки CSV-датасета перед обучением с JSON-отчётом
Docker, Docker Compose	Воспроизводимый учебный стенд: источник, хранилище, приложение, инференс
prometheus_client, JSON-логи, Sentry	Технические метрики, сквозной `request_id`, опциональный сбор ошибок
pytest, Playwright, Ruff, GitHub Actions	Unit, интеграционные, Docker и браузерные проверки в CI

3. ML-эксперименты и результаты

Данные (DATASET.md). Генератор с seed 42 создаёт синтетическую историю трёх таблиц (`users`, `orders`, `events`) с шагом 15 минут: 8 061 наблюдение, временное разбиение train 4 029 / validation 2 016 / test 2 016. Train содержит только нормальный режим. В validation и test по 9 размеченных инцидентов трёх типов: всплеск NULL, скачок объёма и смена режима. Дельты считаются только по предыдущему наблюдению того же ряда; scaler обучается только на train. Перед обучением выполняется валидация (20 проверок, раздел 4).

Протокол (PROTOCOL.md) зафиксирован в Git до запусков (коммит `f94947c`). Пять конфигураций отличаются от baseline одним параметром. Правило выбора задано заранее: среди конфигураций, выполнивших все три цели на validation, взять максимальный F1 по наблюдениям. Test оценивается один раз, только для выбранной конфигурации, отдельным шестым run `final_test`. Precision, recall и F1 считаются по наблюдениям; обнаружение инцидента — хотя бы один сигнал внутри его интервала.

Run (validation)	contamination / деревья / max_samples	Precision	Recall	F1	FPR	Инциденты	Цели
baseline	0,01 / 100 / 256	35,22 %	42,03 %	**0,3833**	8,84 %	9/9	да
contamination_005	0,005 / 100 / 256	38,00 %	27,54 %	0,3193	5,14 %	9/9	да
contamination_050	0,05 / 100 / 256	27,73 %	82,13 %	0,4146	24,49 %	9/9	нет
trees_300	0,01 / 300 / 256	35,53 %	39,13 %	0,3724	8,13 %	9/9	да
samples_128	0,01 / 100 / 128	32,79 %	38,65 %	0,3548	9,07 %	9/9	да

Таблица 1. Пять runs MLflow на validation: 2 016 наблюдений, 1 809 нормальных. Медиана задержки резких инцидентов — 0 или 0,5 цикла у всех вариантов. Источник: RESULTS.md, comparison.csv, summary.json.

Сравнение. Наибольшее влияние из исследованных параметров оказывает `contamination`. Повышение до 0,05 даёт лучший F1 (+0,031), но FPR растёт до 24,49 % — конфигурация исключена правилом. Снижение до 0,005 уменьшает FPR на 3,7 п. п. ценой F1 −0,064. 300 деревьев и `max_samples=128` F1 не улучшили. Все варианты нашли 9/9 инцидентов, поэтому полнота по событиям их не различает. Выводы локальные: один seed и однофакторные изменения.

Выбрана baseline (0,01 / 100 / 256) — максимальный F1 среди допустимых. Результат на test:

Метрика `final_test` (baseline)	**Результат**	**Цель**
Обнаруженные инциденты	9/9 (100 %)	≥ 80 % — выполнена
Ложные сигналы вне инцидентов (FPR)	**240/1 809 (13,27 %)**	≤ 10 % — **не выполнена**
Медиана задержки резких инцидентов	0 циклов	≤ 1 — выполнена
Precision / recall / F1 по наблюдениям	27,71 % / 44,44 % / 0,3414	не задана
TP / FP / FN / TN	92 / 240 / 115 / 1 569	2 016 наблюдений

Таблица 2. Единственная оценка на test, run `36db7cd846c94d05897732aa3eec448d`.

Вывод. Модель **не прошла критерий допуска по ложным сигналам**, поэтому реестр не допускает её как рабочую. FPR вырос с 8,84 % на validation до 13,27 % на более позднем test. Гипотеза: абсолютный `row_count` растёт со временем и выходит из диапазона обучения; это не доказано. Медиана скрывает поздние сигналы: скачок объёма `users` найден через 5 циклов (75 минут), `orders` — через 2. Выбор после раскрытия test не менялся. Подбор `contamination=0,005` задним числом по test запрещён протоколом. Полный комплект — MLflow-экспорт с пятью runs, `final_test`, моделями, предсказаниями и инструкцией импорта (README); аудит по таблицам и типам — quality/README.md.

Отдельная demo-модель стенда. Модель #9 для таблицы `events` обучена на `row_count` 56–68 тыс. На Compose-стенде с таблицей 2 000–5 200 строк она дала одинаковый score −0,077 всем точкам: векторы вне области обучения. Поэтому для стенда по отдельному протоколу, зафиксированному до test, на синтетическом режиме стенда заново обучена та же сетка из пяти конфигураций; по правилу #9 на validation выбрана `contamination_005` (demo-model/RESULTS.md). На её собственном test: FPR 1,66 % (52/3 133), инциденты 440/440, сценарий Compose без ошибок в 190 из 200 эпизодов. Эта модель помечена `demo=true`, `quality_admitted=false`, не заменяет результат #9 и не является рабочей моделью.

4. Качество данных и тестирование

Валидация данных (validation/README.md). Скрипт на pandas проверяет манифест и SHA-256 файлов, схему CSV, типы и конечность, диапазоны (`null_rate` ∈ [0, 1], `row_count` ≥ 0), UTC-время, уникальность, порядок, непересечение выборок, полноту временной сетки, согласованность дельт и разметку. На исходном наборе — 20 проверок пройдено, 0 провалено. На намеренно испорченной копии (пустой идентификатор, текст вместо числа, выход за диапазон, время без UTC, дубликаты, перестановки и др.) — 14 проверок провалено. При ошибке обучение не начинается.

Тест-план (testing/STRATEGY.md) построен по рискам:

Уровень	**Что проверяет**	**Где выполняется**
Unit и контракт	Выравнивание и дельты признаков, пустая история, OpenAPI-контракты, изоляция проектов, LLM- и Telegram-моки	CI: Python 3.11, 3.12, 3.13
Интеграция модели	Настоящие IsolationForest, scaler и joblib с временной SQLite → `/api/anomalies`	в обычном наборе
Поведение модели (#10)	13 тестовых случаев: стабильный ряд, всплеск NULL, скачок объёма, смена режима, шум, пропуски, повторный scoring без дообучения	в обычном наборе
Docker-интеграция	PostgreSQL, MySQL, ClickHouse, TimescaleDB, Iceberg; сборка образа `--no-cache`; Compose-сценарий	CI `integration`
E2E	Chromium → настоящий Flask	CI, только push в master

Команда `check_model_quality` пересчитывает метрики из сохранённого архива #9 и для baseline **ожидаемо завершается кодом 1 (FAIL)**: зелёный pytest не означает допуск модели (quality/README.md). Проверка тест-плана в CI PR #40 (`39db8f6`): три `test`, `integration` и `ruff` — SUCCESS; локально 1 239 passed, 6 failed из-за окружения (нет `sqlite3` CLI и `python` в PATH), разбор — в VERIFICATION.md.

Выявленные проблемы и риски.

- **Высокий FPR baseline** — 13,27 % на test (раздел 3). Допуск не выдан, критерий не ослаблен.
- **Модель вне области обучения.** На CI архивная модель #9 пометила аномалией все нормальные тики стенда: масштаб таблиц стенда далёк от данных обучения. При разборе найден и независимый дефект: генератор `live_demo` не заполнял две колонки и поднимал `null_rate` в «норме». Генератор исправлен, для стенда обучена отдельная demo-модель (demo-model/PROTOCOL.md).
- **Вероятностная нестабильность demo-модели.** Нормальный поток стенда генерируется без фиксированного зерна, и по test сценарий проходит в 190/200 случаев (95 % ДИ 0,91–0,97): примерно один прогон из 20 даёт ложную тревогу. Это наблюдалось в PR #46: в run 42674706790 job `integration` упал на `check-outcome` (`false_alarms 1` из 3 нормальных тиков, инцидент найден), повторная попытка на том же HEAD `62b9f4a` прошла. Это ограничение качества модели, а не неисправность системы. Один успешный повтор не доказывает отсутствие нестабильности; порог и зерно ради PASS не подбирались.
- **Нестабильность самой проверки.** Окно наблюдения Compose-сценария открывалось с микросекундами, а метки метрик хранятся с точностью до секунды, из-за чего тик мог выпасть (HEAD `d8f65f4`). Исправлено, добавлены регрессионные тесты (containers/verification.md).

5. Деплой и мониторинг

Маршрут запуска (containers/README.md). `Dockerfile` и `docker-compose.yml` поднимают PostgreSQL-источник, TimescaleDB-хранилище и Flask-приложение со сборщиками; профиль `inference` добавляет FastAPI на том же образе с реестром только для чтения. `scripts.init_compose_demo` создаёт `.env` без внешних ключей (NIM и Telegram пусты), порты публикуются только на `127.0.0.1` (Flask 15001, FastAPI 18000). Далее — seed демонстрационной БД, обучение demo-модели, запуск инференса и `curl /healthz`. CI выполняет этот маршрут из чистого checkout: сборка `nocache`, readiness, нормальные тики, инцидент, перезапуск БД с проверкой сохранности данных. Docker в среде разработки отсутствовал, поэтому Compose проверялся только в CI. Независимый ручной прогон человеком, не участвовавшим в разработке, **не выполнен** (критерий 7 в verification.md).

Стратегия релиза и отката (deployment/STRATEGY.md, #14) — **проектное решение**. Для приложения целевой подход — blue-green. На стенде нет двух сред и переключателя трафика, а планировщик стартует в каждом процессе Flask. Поэтому предложено проверять green в изолированном Compose-проекте на копии данных и затем заменять контейнер между тиками. Для модели — shadow, затем canary по источникам через привязки версий. Реализованы CI, реестр с допуском, привязки и оценки с ключом версии, алерты. `release.yml` и runbook отката есть в коде, но не запускались. Переключатель трафика, shadow-оценка и откат **не реализованы и не испытаны**. Этап допуска модели сейчас заканчивается «СТОП»: допущенной модели нет.

Технические метрики (monitoring/README.md, #15): `/healthz` с пробами зависимостей; `/metrics` в формате Prometheus — `http_requests_total`, гистограмма `http_request_duration_seconds` (p95), `collector_runs_total`; JSON-логи с `request_id`; маскирование секретов, в том числе в traceback (дефект найден и исправлен в #15). Эскиз дашборда из 10 панелей — DASHBOARD.md. Prometheus-сервер и Grafana не развёрнуты.

Показатели данных и модели (monitoring/model/README.md, #16) считаются по одному источнику и одной версии. Входы: пропуски и невалидные значения каждого признака, распределения, свежесть, признак достоверности вывода `conclusive`. Модель: распределение score, доля аномалий, версия и допуск, ошибки инференса `inference_predictions_total{outcome}`, время расчёта (`Server-Timing`). Текущий поток не размечен, поэтому precision/recall по нему не считаются. **Дрейф** (drift/STRATEGY.md, #17):

72-часовое окно против 14-суточного эталона; KS: D ≥ 0,10 и p < 0,0125; PSI > 0,20 — предупреждение, > 0,25 — сильный сдвиг. Для растущего `row_count` используется коридор тренда. Переобучение — по событию с ручным допуском; эти правила описаны и проверены синтетическим сценарием, но не автоматизированы.

Алерт	Условие	Проверка
`HttpErrorRatioHigh`	Доля 5xx > 10 % за 5 минут при ≥ 10 запросах	Сценарий «норма → сбой БД → восстановление»: `firing` на 4-м тике сбоя, `resolved` на 7-м тике восстановления; повтор против живого сервера (demo.json)
`ModelAnomalyRateHigh`	≥ 3 аномалий среди 20 последних результатов ряда; порог выведен из baseline 10/1 175 на validation demo-модели	`firing` на тике 33, `resolved` на тике 54, 19 проверок `true` (demo.json)

Таблица 3. Алерты пишутся в журнал `app.alerts`. Доставка во внешний канал (Telegram, почта) не реализована.

6. Использование ИИ-ассистентов

Источник — журнал ИИ-ассистентов (#18); исходные диалоги и промпты не сохранялись и в отчёте не реконструируются. Записи журнала заканчиваются 08.11.2026; более поздние работы в журнал не внесены и ниже не оцениваются.

Задачи	Результат ассистента	Проверка и исправления
План курса, Issues с критериями и зависимостями (#18)	Аудит пробелов, ROADMAP, задачи	Человеческая проверка в журнале не подтверждена
Датасет (#7), валидация (#8), эксперименты (#9)	Генератор, валидатор, протокол, 6 runs MLflow, экспорт	Тесты и Ruff в PR; выбор модели после test не менялся; PR слиты
Проверка качества модели (#10)	Поведенческие тесты, аудит архива, отрицательный допуск	Ревью Раиля 14.10: исправлены границы инцидента (исключён тик восстановления), закреплён SHA-256 архива
Архитектура и ADR (#3)	Схема, ADR Flask/FastAPI, ограничения со ссылками на код	Ревью Раиля 14–15.10: уточнены источник активной модели, откат, требование миграции ключа scores
Журнал и аудит LLM-объяснений (#18)	Анализ пути запроса и данных, тесты	Отдельная проверка другим ассистентом нашла 6 мелких замечаний, все исправлены; официальный APPROVED Оксаны 25.10
Стратегия дрейфа (#17)	Стратегия, PDF, синтетический сценарий	Ревью Раиля 26.10: доработаны проверка значений до KS/PSI и обработка растущего `row_count`

Инструменты по журналу — Codex (модель не установлена) и Claude Code (Claude Opus 5.5, доработка #17). Принцип проверки: результат ассистента принимается только с воспроизводимой командой, тестом или артефактом и ревью другого участника. Выявленные недостатки: неверная граница интервала инцидента в метриках, неполные формулировки ADR, неучтённые невалидные значения и рост `row_count` в стратегии дрейфа. Ограничение: журнал фиксирует задачи и проверки, но не промпты, и поэтому не покрывает отчёт B15.

LLM внутри продукта — отдельный компонент: формулирует гипотезу о причине аномалии (LLM_LIMITATIONS.md). Решение об аномалии принимает Isolation Forest. Аудит 24.10.2026 показал риски: `confidence` не калиброван, имена таблиц и колонок попадают в промпт без защиты от prompt injection, ответ не проверяется на истинность. Кэш объяснений не учитывал проект — исправлено в #12; остальные семантические ограничения сохраняются. При отсутствии ключа или сбое NIM работает шаблонный ответ.

7. Ограничения, выводы и развитие

Существенные ограничения.

- **Синтетические данные.** Все метрики качества получены на сгенерированных наборах: 9 инцидентов в test #9, отдельный режим demo-модели. Перенос на реальные БД не доказан.

- **Модель.** Baseline не выполнила цель FPR ≤ 10 % (13,27 %); demo-модель стенда допуска не имеет и вероятностно даёт ложную тревогу примерно в 1 сценарии из 20. Четыре признака чувствительны к росту абсолютного объёма таблиц.
- **Дрейф.** Рост таблиц выводит векторы из области обучения, что уже наблюдалось на стенде. Пороги KS/PSI — начальные настройки, их эффективность на реальном потоке не измерена. Автоматического переобучения нет.
- **Границы автоматизации.** Модель и LLM дают сигнал и гипотезу, решение принимает специалист. Алерты не доставляются во внешний канал; Prometheus-сервера нет.
- **Деплой.** Blue-green, shadow/canary и откат описаны, но не испытаны; `release.yml` не запускался; независимый ручной прогон стенда не выполнен.
- **Прототип, не production.** Планировщик внутри процесса Flask без распределённой блокировки, нет HA и очередей, часть legacy-маршрутов API сохраняет прежний режим доступа.

Вывод. На базе DB Monitoring построен воспроизводимый учебный контур ИИ-компонента. В него входят: синтетический датасет с разметкой и временным разбиением, валидация из 20 проверок, пять зафиксированных экспериментов MLflow с одной честной оценкой на test, тесты модели и CI, FastAPI-сервис с реестром версий, Compose-стенд, проверенный в CI, технический алерт и алерт модели. Главный результат по качеству отрицательный, и он зафиксирован: модель находит все 9 инцидентов test с медианной задержкой 0 циклов, но даёт 13,27 % ложных сигналов и потому не допущена к эксплуатации.

Предложения по развитию (не выполнены): новая заранее объявленная серия экспериментов с признаками, устойчивыми к росту таблиц, и новым закрытым test; оценка на реальной истории с подтверждёнными инцидентами; доставка алертов и Prometheus-сервер; испытание отката по runbook; независимый прогон стенда другим участником.

Материалы проекта: план и задачи курса · описание кейса · Issues · эксперименты MLflow · качество модели · Compose-стенд · мониторинг · журнал ИИ-ассистентов.